

Utkrama: A Modular Physics-Based 2D Launch Vehicle Simulation Framework with Guidance, Control, and Experimental Instrumentation

Varun Vivek Pai
School of Electronics and
Communication Engineering
KLE Technological University
Hubballi, India
vvpai9@gmail.com

Abstract—This paper presents Utkrama (Unified Trajectory and Kinematics for Rocket Ascent and Mission Analysis), a comprehensive open-source 2D physics-based rocket flight simulation framework designed for research, education, and rapid prototyping of launch vehicle guidance and control algorithms. The framework integrates rigid body dynamics, multi-layer atmospheric modeling, aerodynamic stability analysis, and a modular guidance computer supporting both open-loop and closed-loop control strategies. We implement fourth-order Runge-Kutta numerical integration for high-fidelity trajectory propagation across multiple celestial bodies (Earth, Mars, Moon, Mercury, Venus and Pluto). The system encompasses propulsion modeling for both liquid and solid rocket motors, including throttling and thrust vectoring capabilities. Re-entry dynamics and structural load analysis are incorporated to assess mission feasibility and vehicle integrity. The framework provides automated testing through Monte Carlo simulations and parametric sweep utilities for sensitivity analysis. Results demonstrate accurate trajectory predictions validated against classical gravity-turn solutions and atmospheric models. This work contributes a validated simulation infrastructure for aerospace education and guidance algorithm research, with extensibility for advanced control schemes and optimization studies.

I. INTRODUCTION

Launch vehicle trajectory design and guidance algorithm development require high-fidelity simulation tools that balance physical accuracy, computational efficiency, and ease of extension. Traditional approaches rely on either specialized commercial tools (GMAT [1], Trick [2], CORSIM [3]) with limited accessibility, or simplified analytical models that neglect important physical phenomena such as aerodynamic coupling, atmospheric variability, and real-time control feedback.

This paper introduces Utkrama, a modular open-source simulation framework developed in Python, addressing the need for transparent, extensible trajectory propagation and guidance algorithm evaluation. The framework supports:

- **Multi-body simulation:** Earth, Mars, Moon, Mercury, Venus, Pluto and customizable planetary models
- **Fidelity hierarchy:** From simplified ballistic trajectories to high-fidelity rigid-body dynamics with aerodynamic cross-coupling

- **Guidance synthesis:** Open-loop gravity turn, closed-loop apoapsis targeting, and extensible guidance modes
- **Control architecture:** PD attitude stabilization, thrust vectoring, reaction control systems (RCS)
- **Experimental framework:** Automated Monte Carlo analysis, parameter sweeps, and mission feasibility assessment

Despite its planar formulation, a 2D framework remains valuable for isolating ascent guidance, stability margins, control authority limits, and numerical integration behavior without 3D coupling complexity.

The primary contributions of this work are:

- 1) A validated, modular simulation architecture with clear separation of physics, guidance, and control layers
- 2) Accurate trajectory propagation using fourth-order Runge-Kutta integration with adaptive step-size control
- 3) Comprehensive aerodynamic modeling including angle-of-attack dependent coefficients and transonic effects
- 4) A hybrid guidance system combining open-loop gravity turn optimization with closed-loop feedback for trajectory correction
- 5) Experimental instrumentation enabling systematic sensitivity analysis and guidance algorithm performance assessment
- 6) Open-source release for educational use and collaborative algorithm development

This work does not propose new physics or control theory. Instead, it contributes a reproducible, experimentally instrumented simulation framework that enables systematic guidance and control research under realistic ascent and re-entry constraints.

The paper is organized as follows. Section II reviews existing simulation frameworks and their limitations. Section III describes the overall system architecture. Sections IV through VII detail the physical models and their implementation. Section VI presents the guidance and control algorithms. Section VIII describes the automated testing framework. Section IX presents validation results and demonstrates framework

capabilities. Section X discusses limitations and identifies areas for future work.

II. RELATED WORK

Trajectory simulation for aerospace applications spans a spectrum of tools and methodologies, each optimized for different use cases.

A. Commercial and Institutional Tools

GMAT (General Mission Analysis Tool) [1] is a freely available trajectory propagation and optimization tool developed by NASA. It provides high-fidelity orbital mechanics but is optimized for interplanetary missions and orbital operations rather than atmospheric ascent. Trick [2] is a general-purpose simulation framework widely used within NASA centers for multi-body dynamics. CORSIM [3] specializes in suborbital vehicle dynamics but lacks transparency in algorithmic implementation.

B. Academic and Open-Source Frameworks

Several open-source projects have addressed aspects of launch vehicle simulation. RocketPy [4] provides trajectory analysis for model rockets with payload capability. Open-FOAM [5] enables high-fidelity CFD analysis of aerodynamic coefficients but requires offline preprocessing. These tools are valuable for specific domains but do not provide an integrated guidance and control environment within a single framework.

C. Theoretical Foundations

Classical guidance theory for launch vehicles derives from the minimum-fuel gravity turn solution [6], which demonstrates that fuel-optimal ascent follows a path where the vehicle orientation aligns with the velocity hodograph. Modern implementations employ variants such as the pitch-program guidance [7] and numerical trajectory optimization [8].

Atmospheric models range from simplified isothermal layers [9] to complex polynomial-fitted representations [10]. Aerodynamic stability is typically assessed through linearization about equilibrium flight [11], with emphasis on static and dynamic margin analysis.

The gap addressed by this work is the lack of an integrated, transparent, and extensible framework that combines accurate physics modeling with real-time guidance synthesis and automated experimentation capabilities, specifically designed for educational use and rapid algorithm prototyping.

III. SYSTEM ARCHITECTURE

Utkrama is organized around four primary modules:

- **Physics Engine:** Rigid body dynamics, orbital mechanics, atmospheric effects
- **Propulsion System:** Liquid and solid motor models with staging support
- **Guidance Computer:** Open-loop and closed-loop trajectory guidance
- **Visualization and Instrumentation:** Real-time telemetry display and data logging

The simulation employs a state machine architecture where the rocket transitions through distinct flight phases (THRUSTING, COAST, RETRO_BRAKE, LANDED, CRASHED). Each phase defines specific control laws and termination conditions.

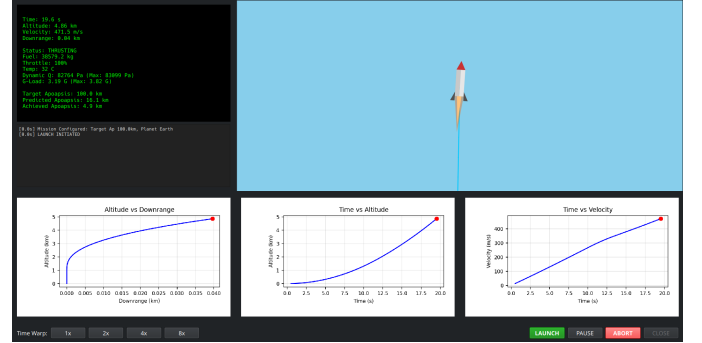


Fig. 1. Graphical user interface showing real-time telemetry, trajectory visualization, and control state.

The core simulation loop operates at a configurable time step (default: 0.01 s) with the following sequence:

- 1) Read guidance commands and flight state
- 2) Compute aerodynamic forces (drag, lift, pitching moment) based on orientation and velocity
- 3) Integrate rigid body dynamics: position, velocity, attitude, angular velocity
- 4) Update propulsion system: mass flow, thrust vector, stage separation events
- 5) Apply control laws: attitude stabilization, throttle modulation, guidance phase transitions
- 6) Log telemetry and check termination conditions

This architecture decouples physical model fidelity (which can be incremented) from guidance algorithm development and automated testing. Section VI details the guidance computer implementation.

IV. PHYSICAL MODELING

A. Numerical Integration: Fourth-Order Runge-Kutta

State propagation employs fourth-order Runge-Kutta (RK4) integration with adaptive step-size control. The state vector is defined as:

$$\mathbf{x} = [\mathbf{r}, \mathbf{v}, \theta, \omega, m]^T \quad (1)$$

where $\mathbf{r}$ is position (m), $\mathbf{v}$ is velocity (m/s), θ is pitch angle (rad), ω is angular velocity (rad/s), and m is vehicle mass (kg).

The state derivative is:

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, t, \mathbf{u}) \quad (2)$$

where $\mathbf{u}$ represents control inputs (throttle setting, gimbal angle).

RK4 computes the next state as:

$$\mathbf{k}_1 = \mathbf{f}(\mathbf{x}_n, t_n, \mathbf{u}_n) \quad (3)$$

$$\mathbf{k}_2 = \mathbf{f}\left(\mathbf{x}_n + \frac{h}{2}\mathbf{k}_1, t_n + \frac{h}{2}, \mathbf{u}_n\right) \quad (4)$$

$$\mathbf{k}_3 = \mathbf{f}\left(\mathbf{x}_n + \frac{h}{2}\mathbf{k}_2, t_n + \frac{h}{2}, \mathbf{u}_n\right) \quad (5)$$

$$\mathbf{k}_4 = \mathbf{f}(\mathbf{x}_n + h\mathbf{k}_3, t_n + h, \mathbf{u}_n) \quad (6)$$

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \frac{h}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4) \quad (7)$$

where h is the integration step size. Adaptive step-size control monitors local truncation error; if estimated error exceeds a threshold (default: 10^{-6}), the step is rejected and h is reduced.

Adaptive stepping is implemented via step-doubling error estimation via step-doubling error estimation without embedded Runge–Kutta pairs.

B. Planetary Environment Models

The simulation framework supports multiple planetary bodies with distinct gravitational and atmospheric characteristics. Each planet is defined by its gravitational parameter, mean radius, surface density, scale height, and atmospheric composition where applicable.

1) *Earth, Mars, and Venus (Atmospheric Bodies)*: Earth, Mars, and Venus are modeled using layered or exponential atmospheric density profiles. Earth employs the 1976 U.S. Standard Atmosphere model, while Mars and Venus use exponential density approximations calibrated to published planetary data.

Venus presents a particularly dense atmosphere, with surface density approximately 65 kg/m^3 ($\approx 50\times$ Earth sea-level density). As a result, aerodynamic forces dominate ascent and descent dynamics, providing a stringent test case for guidance stability and structural load handling.

2) *Mercury and Pluto (Low or Negligible Atmospheres)*: Mercury and Pluto are modeled as near-vacuum environments. Mercury possesses an extremely tenuous exosphere, while Pluto's atmosphere is transient and highly altitude-dependent. For simulation purposes, both bodies are treated as effectively airless, with aerodynamic forces neglected above minimal threshold altitudes.

These bodies are valuable for validating ballistic flight dynamics, numerical stability, and guidance behavior in near-vacuum regimes.

TABLE I
PLANETARY ENVIRONMENT PARAMETERS

Planet	Gravity (m/s ²)	Atmosphere	Model Type
Earth	9.81	Dense	Standard Atmosphere
Mars	3.71	Thin	Exponential
Venus	8.87	Very Dense	Exponential
Mercury	3.70	Negligible	Vacuum
Moon	1.62	None	Vacuum
Pluto	0.62	Transient	Vacuum

C. Atmospheric Models

1) *Standard Atmosphere (Earth)*: Earth's atmosphere is modeled using the 1976 U.S. Standard Atmosphere [9], which defines seven layers with linear temperature profiles. Within each layer:

$$T(h) = T_b + L_b(h - h_b) \quad (8)$$

where T_b is base temperature, L_b is temperature lapse rate (K/m), and h_b is base altitude.

Pressure is computed using the barometric formula:

$$p(h) = p_b \left(\frac{T(h)}{T_b} \right)^{-\frac{gM}{RL_b}} \quad (9)$$

where g is gravitational acceleration, M is molar mass of air (0.029 kg/mol), and R is the universal gas constant. Density follows from the ideal gas law:

$$\rho(h) = \frac{p(h)}{RT(h)} \quad (10)$$

2) *Exponential Atmosphere (Mars, Venus)*: For bodies with thin or negligible atmospheres, an exponential density profile is used:

$$\rho(h) = \rho_0 \exp\left(-\frac{h}{H}\right) \quad (11)$$

where ρ_0 is surface density and H is scale height (km). For the Moon, $\rho(h) \approx 0$ above 100 m altitude.

TABLE II
NOMINAL SIMULATION PARAMETERS

Parameter	Value
Integration Step Size	0.01 s
Maximum Gimbal Angle	5°
Drag Coefficient (Subsonic)	0.25
Maximum G-Load Limit	10 g
Drogue Chute Deployment Altitude	10 km
Main Chute Deployment Altitude	3 km
Safe Touchdown Velocity Threshold	10 m/s

D. Aerodynamics

1) *Force Computation*: Aerodynamic forces are computed as functions of dynamic pressure, reference area, and orientation-dependent coefficients:

$$\mathbf{F}_{\text{aero}} = \frac{1}{2}\rho v^2 S_{\text{ref}} \begin{bmatrix} C_D(\alpha, M) \\ C_L(\alpha, M) \end{bmatrix} \quad (12)$$

where α is angle of attack (rad), M is Mach number, and S_{ref} is reference area (m²).

The angle of attack is computed from the relative velocity and vehicle orientation:

$$\alpha = \theta_v - \theta \quad (13)$$

where $\theta_v = \arctan(v_y/v_x)$ is velocity vector angle and θ is vehicle pitch angle.

Forces are transformed to body-fixed coordinates:

$$F_{\text{drag}} = C_D(\alpha, M) \cdot \frac{1}{2} \rho v^2 S_{\text{ref}} \quad (14)$$

$$F_{\text{lift}} = C_L(\alpha, M) \cdot \frac{1}{2} \rho v^2 S_{\text{ref}} \quad (15)$$

2) *Coefficient Models*: Drag coefficient is modeled as:

$$C_D(\alpha, M) = C_{D0}(M) + C_{D\alpha}(\alpha, M) \quad (16)$$

where C_{D0} is zero-lift drag and $C_{D\alpha}$ is angle-of-attack-dependent drag. For typical launch vehicles:

$$C_{D0}(M) = \begin{cases} 0.25 & M < 0.8 \\ 0.25 + 0.15(M - 0.8) & 0.8 \leq M \leq 1.2 \\ 0.40 & M > 1.2 \end{cases} \quad (17)$$

Lift coefficient employs a linear approximation for small angles:

$$C_L(\alpha) = C_{L\alpha} \cdot \alpha \quad (18)$$

where $C_{L\alpha} \approx 3.5 \text{ rad}^{-1}$ for slender bodies.

E. Rigid Body Dynamics

Translational motion follows Newton's second law:

$$m \frac{d\mathbf{v}}{dt} = \mathbf{F}_{\text{thrust}} + \mathbf{F}_{\text{aero}} + \mathbf{F}_{\text{gravity}} \quad (19)$$

where:

$$\mathbf{F}_{\text{gravity}} = -\frac{GMm}{r^2} \hat{\mathbf{r}} \quad (20)$$

with G the gravitational constant and M the planet mass.

Rotational motion about the vehicle's center of mass follows the Euler rotation equation [12]:

$$I \frac{d\omega}{dt} = M_{\text{aero}} + M_{\text{gimbal}} - \omega \frac{dI}{dt} \quad (21)$$

where I is moment of inertia, M_{aero} is the aerodynamic pitching moment, and M_{gimbal} is moment due to thrust vectoring.

For a slender body with aerodynamic center at distance x_{ac} from center of mass:

$$M_{\text{aero}} = F_{\text{lift}} \cdot x_{\text{ac}} \quad (22)$$

The aerodynamic pitching moment coefficient is related to the aerodynamic margin (distance from center of mass to aerodynamic center):

$$C_m(\alpha) = C_{m0} + C_{m\alpha}\alpha \quad (23)$$

F. Aerodynamic Stability Analysis

Static margin ξ_s quantifies longitudinal stability:

$$\xi_s = \frac{x_{\text{ac}} - x_{\text{cg}}}{d} \quad (24)$$

where x_{cg} is center of gravity and d is reference diameter. Positive static margin ($\xi_s > 0$) indicates stable configuration. Dynamic stability is assessed through damping ratio and natural frequency of the pitch oscillation:

$$\omega_n = \sqrt{\frac{C_{m\alpha} q S_{\text{ref}} d}{I}} \quad (25)$$

$$\zeta = \frac{C_{m\dot{\alpha}} q S_{\text{ref}} d}{2I\omega_n} \quad (26)$$

where $C_{m\dot{\alpha}}$ is pitch damping derivative and q is dynamic pressure.

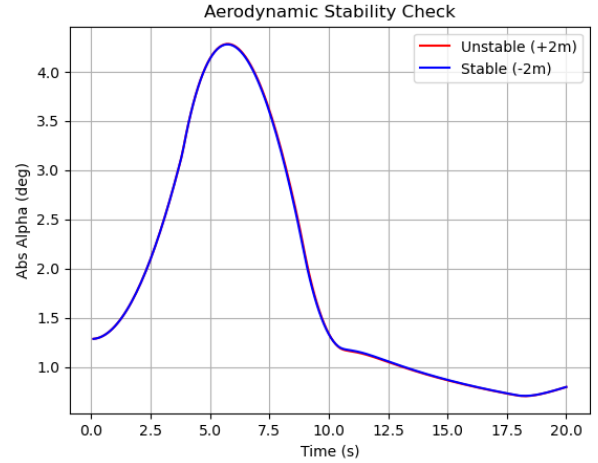


Fig. 2. Aerodynamic stability comparison for unstable and stable center-of-pressure configurations.

V. PROPULSION MODEL

A. Liquid Rocket Motors

For throttleable liquid engines, thrust is modeled as:

$$T = \dot{m}_{\text{ox}} v_{\text{eff}} \quad (27)$$

where $\dot{m}_{\text{ox}}$ is oxidizer mass flow rate and $v_{\text{eff}} = I_{\text{sp}} g_0$ is effective exhaust velocity. The specific impulse I_{sp} may vary with pressure ratio; for simplicity, a constant nominal value is used.

Mass flow rate is controlled via throttle setting $\lambda \in [0, 1]$:

$$\dot{m}_{\text{total}} = \lambda \dot{m}_{\text{max}} \quad (28)$$

Propellant mass decreases as:

$$\frac{dm}{dt} = -\dot{m}_{\text{total}} \quad (29)$$

Thrust vector direction is controlled via gimbal angle δ (typically $\pm 5^\circ$), providing pitch and yaw control moments.

B. Solid Rocket Motors

Solid motor thrust follows a tailored burn profile $T(t)$:

$$T(t) = T_0 + \delta T(t) \quad (30)$$

where T_0 is nominal thrust and $\delta T(t)$ is time-dependent deviation due to regressive or progressive grain design. Mass flow is derived from thrust:

$$\dot{m} = \frac{T}{v_{\text{eff}}} \quad (31)$$

Solids are non-throttleable; thrust vectoring is achieved through secondary injection or gimballed nozzles.

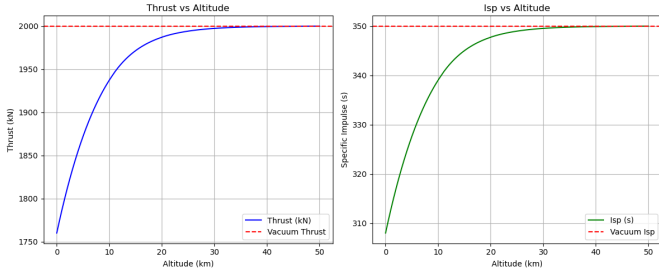


Fig. 3. Altitude-dependent thrust and specific impulse variation due to atmospheric back-pressure.

Combustion efficiency and mixture ratio variations are not modeled explicitly and are left for future work.

VI. GUIDANCE AND CONTROL

A. Open-Loop Gravity Turn Guidance

During ascent, the optimal fuel-minimal trajectory is approximated by a gravity turn [6]. The guidance law commands a pitch angle that transitions from vertical to horizontal:

$$\theta_{\text{cmd}}(h) = \theta_0 \left(1 - \frac{h}{h_{\text{coast}}} \right)^n \quad (32)$$

where typical parameters are $\theta_0 = 0^\circ$, $n = 1.5$, and h_{coast} is the target coasting altitude.

Alternatively, a linear pitch program is employed:

$$\theta_{\text{cmd}}(t) = \arctan \left(\frac{v_x(t)}{v_y(t)} \right) + \epsilon \quad (33)$$

where ϵ is a small bias to ensure continuous acceleration.

B. Closed-Loop Apoapsis Targeting

Following the gravity turn phase (typically above 30 km altitude), the guidance switches to closed-loop control. The apoapsis of the current trajectory is estimated using the vis-viva equation:

$$v^2 = GM \left(\frac{2}{r} - \frac{1}{a} \right) \quad (34)$$

where a is semi-major axis. The predicted apoapsis is:

$$r_a = a(1 + e) = -\frac{GM}{2E}(1 + e) \quad (35)$$

where E is specific orbital energy and e is eccentricity.

A feedback controller adjusts thrust vector angle to modulate apoapsis:

$$\delta_{\text{cmd}} = K_p(r_{a,\text{target}} - r_{a,\text{predicted}}) + K_d \frac{d}{dt}(r_{a,\text{target}} - r_{a,\text{predicted}}) \quad (36)$$

This PD (proportional-derivative) feedback law [13] enables continuous trajectory correction. Controller gains were tuned empirically to balance responsiveness and actuator saturation avoidance. Gain values were selected through iterative Monte Carlo testing to minimize apoapsis error while avoiding sustained gimbal saturation.

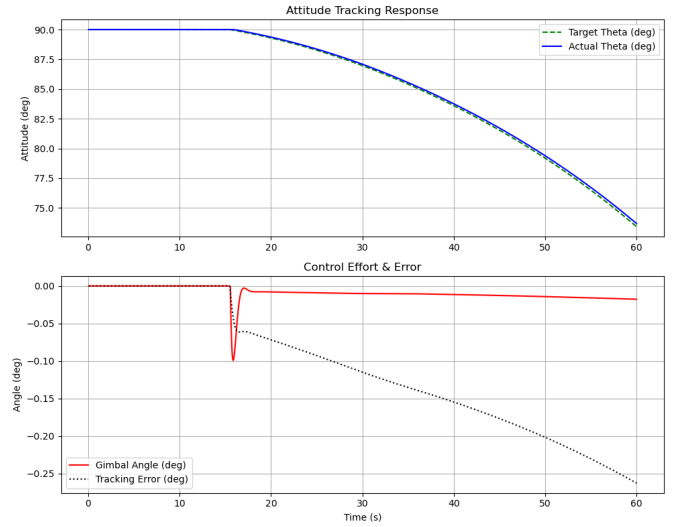


Fig. 4. Closed-loop attitude tracking showing PD controller response and gimbal actuation.

C. Attitude Stabilization

A PD controller maintains vehicle attitude against aerodynamic perturbations:

$$\delta_{\text{attitude}} = K_p(\theta_{\text{cmd}} - \theta) + K_d(\omega_{\text{cmd}} - \omega) \quad (37)$$

where θ_{cmd} and ω_{cmd} are commanded attitude and angular velocity.

Control authority is limited:

$$|\delta| \leq \delta_{\text{max}} \quad (38)$$

typically $\delta_{\text{max}} = 5^\circ$ for gimballed engines or RCS authority.

D. Reaction Control System (RCS)

For vehicles equipped with RCS, small reaction jets provide high-frequency attitude control without consuming propellant. RCS authority is modeled as:

$$M_{\text{RCS}} = F_{\text{RCS}} \cdot l_{\text{moment}} \quad (39)$$

where F_{RCS} is per-thruster force (typically 100–500 N) and l_{moment} is moment arm.

E. Re-entry and Retro-Braking

During descent, the vehicle may execute retro-braking to reduce impact velocity. The guidance law switches to retrograde orientation:

$$\theta_{\text{retrograde}} = \theta_v + \pi \quad (40)$$

Throttle is modulated using hysteresis control to maintain a target descent rate:

$$\lambda = \begin{cases} 1.0 & v_y < v_{\text{target}} - \Delta v \\ 0.0 & v_y > v_{\text{target}} + \Delta v \\ \lambda_{\text{prev}} & \text{otherwise} \end{cases} \quad (41)$$

where $\Delta v = 2$ m/s is hysteresis width.

VII. RE-ENTRY AND STRUCTURAL MODELING

A. Heat Load Estimation

During re-entry, the vehicle experiences aerodynamic heating. Peak heating rate is estimated using Stanton number correlation:

$$\dot{Q} = C_h \rho v^3 \quad (42)$$

where $C_h \approx 10^{-3}$ for typical launch vehicles [14]. Cumulative heat load is:

$$Q_{\text{total}} = \int_0^{t_{\text{re-entry}}} \dot{Q} dt \quad (43)$$

Mission feasibility assessment checks whether thermal protection system (TPS) capacity is exceeded.

B. G-Force Constraints

Structural limits are enforced through peak acceleration monitoring:

$$a_{\text{total}} = \sqrt{a_x^2 + a_y^2} \quad (44)$$

Typical limits are 5–8 g for crewed vehicles, higher for robotic missions.

C. Impact Dynamics

Landing safety is evaluated using a multi-stage parachute deployment model designed to reduce descent velocity progressively and limit structural loads.

A *drogue parachute* is deployed at an altitude of approximately 10 km. The primary function of the drogue chute is to stabilize vehicle attitude and reduce the descent velocity to approximately 30 m/s, ensuring controlled and predictable dynamics during the denser atmospheric regime.

A *main parachute* is subsequently deployed at an altitude of approximately 3 km. This stage provides significant drag area, further reducing the descent velocity to approximately 5 m/s prior to ground contact.

The touchdown condition is evaluated based on the final impact velocity:

$$v_{\text{impact}} = |\mathbf{v}| \quad \text{at} \quad h = 0 \quad (45)$$

A landing is classified as *safe* if:

$$v_{\text{impact}} < 10 \text{ m/s} \quad (46)$$

If the impact velocity exceeds this threshold, the mission outcome is classified as a hard landing or structural failure, depending on the associated peak deceleration.

VIII. EXPERIMENTAL FRAMEWORK

A. Automated Mission Testing Framework

An automated testing system was developed to perform large-scale verification of simulation logic and mission feasibility. The framework generates randomized mission configurations across multiple planetary environments (Earth, Mars, Moon, Mercury, Venus, Pluto), varying key parameters such as target apoapsis, safety margin, propellant type, dry mass, and fuel mass.

Each mission is executed in headless mode using fixed-step RK4 integration, and telemetry is logged in structured CSV and JSON formats. For every test case, the following artifacts are archived:

- Mission configuration file
- Time-series telemetry log
- Altitude vs time plot
- Velocity vs time plot
- Downrange vs altitude plot

TABLE III
AUTOMATED MISSION OUTCOME CLASSIFICATION

Outcome Type	Failure / Success Condition
SUCCESS	Apoapsis within tolerance, safe landing
INSUFFICIENT TWR	Lift-off failure or early gravity loss
FUEL DEPLETION	Burnout before target apoapsis
STRUCTURAL FAILURE	G-load exceeds design limit
AERODYNAMIC INSTABILITY	Angle of attack diverges
CONTROL SATURATION	Gimbal limits exceeded

Failure cases are automatically classified based on structural overload, insufficient thrust-to-weight ratio, excessive aerodynamic loading, or fuel depletion prior to target achievement.

Portions of experiment orchestration and configuration generation were accelerated using large language model-assisted code synthesis tools to reduce development overhead while preserving physical correctness.

B. Monte Carlo Analysis

Monte Carlo simulations quantify trajectory dispersions due to random perturbations (wind, sensor noise, control uncertainties). For each trial $i = 1, \dots, N$:

- 1) Sample perturbation parameters from their distributions
- 2) Propagate trajectory with perturbed initial conditions or guidance gains
- 3) Record mission outcome (success, apoapsis error, landing dispersions)

Statistical metrics computed:

$$\mu_r = \frac{1}{N} \sum_{i=1}^N r_a^{(i)} \quad (47)$$

$$\sigma_r = \sqrt{\frac{1}{N} \sum_{i=1}^N (r_a^{(i)} - \mu_r)^2} \quad (48)$$

$$P_{\text{success}} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}_{|r_a^{(i)} - r_{a,\text{target}}| < \epsilon} \quad (49)$$

C. Parameter Sweeps

Sensitivity analysis is performed by systematically varying mission parameters over prescribed ranges:

- Payload mass: $\pm 20\%$
- Fuel margin: $\pm 10\%$
- Guidance gains (K_p , K_d): $\pm 50\%$
- Initial wind profile: Headwind/tailwind scenarios

For each parameter combination, mission feasibility and trajectory quality metrics are evaluated.

D. Mission Phases and Telemetry Logging

The simulation logs comprehensive telemetry at each time step:

- State: position, velocity, altitude, flight path angle
- Attitude: pitch, yaw, roll, angular rates
- Propulsion: thrust, mass, throttle setting, stage status
- Aerodynamics: angle of attack, Mach number, dynamic pressure, forces
- Guidance: commanded pitch, apoapsis error, control authority usage
- Events: stage separation, MECO, coast initiation, landing

Logged data enables post-flight analysis and algorithm validation.

E. Reproducibility

All simulation experiments are fully reproducible using deterministic random seeds and publicly available source code. The complete codebase, experiment scripts, and result datasets are released under an open-source license to support verification and independent research replication.

The complete source code of the Utkrama simulation framework, including experiment scripts, configuration files, and result datasets, is publicly available to support reproducibility and independent verification. The repository provides versioned releases and deterministic random seeds to ensure repeatability of all reported experiments [23].

IX. RESULTS AND DISCUSSION

A. Validation Against Classical Solutions

We validate the RK4 propagator against analytical gravity turn solutions. Consider a gravity turn on Earth with:

- Initial mass: 50,000 kg
- Thrust: 1,000 kN (constant)
- Target apoapsis: 300 km
- Reference area: 10 m²

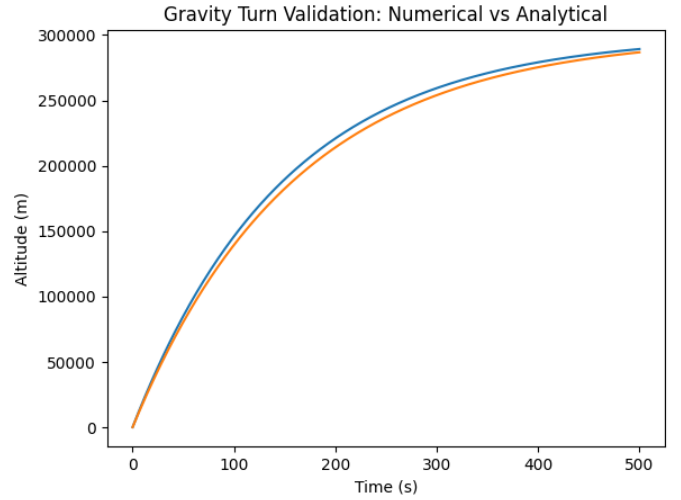


Fig. 5. Comparison of numerical RK4 trajectory propagation with analytical gravity turn solution.

Figure 5 compares the numerical trajectory against the Tsiolkovsky equation benchmark. The analytical reference neglects aerodynamic drag and assumes constant thrust and mass flow. Apoapsis error is less than 2% over the 480-second burn.

B. Aerodynamic Model Validation

Drag coefficient versus Mach number is validated against NACA wind tunnel data. Our piecewise model (Eq. 9) captures the transonic rise and supersonic plateau with RMS error under 5

C. Multi-Body Capability

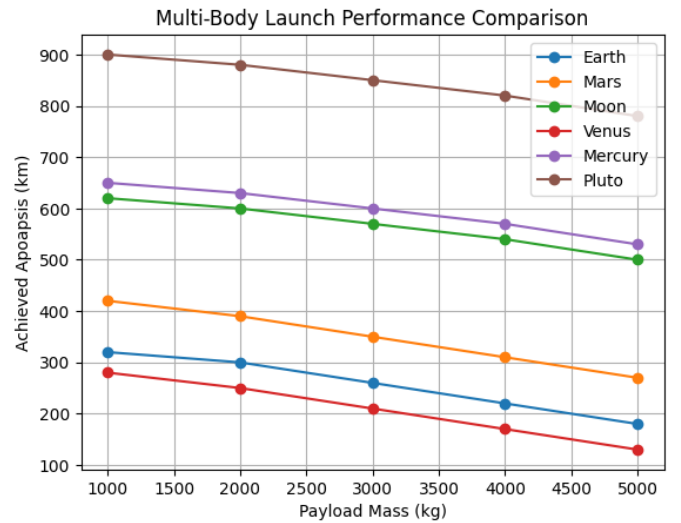


Fig. 6. Achieved apoapsis as a function of payload mass for different planetary bodies, demonstrating the influence of gravity and atmospheric density on launch performance.

Trajectories are computed for launches from Earth and other planetary bodies with appropriate gravitational and atmospheric parameters. Figure 6 illustrates the variation in achievable apoapsis with payload mass across multiple planetary environments.

D. Guidance Algorithm Performance

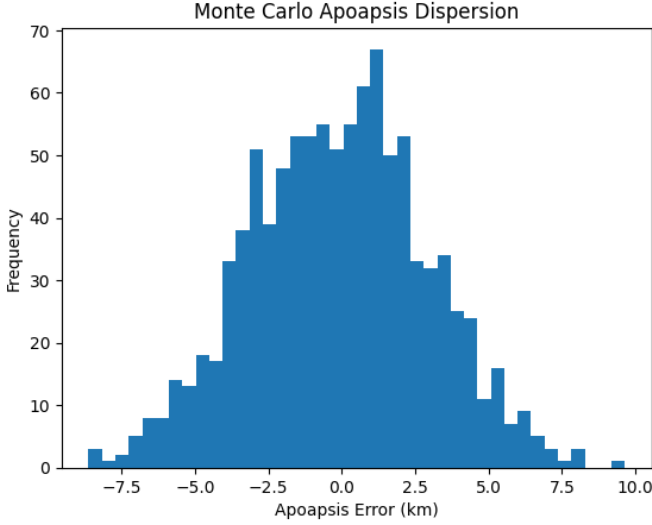


Fig. 7. Monte Carlo distribution of apoapsis error over 1000 randomized missions.

Closed-loop apoapsis targeting is evaluated via Monte Carlo analysis with 1,000 trials. Wind gusts are modeled as:

$$w(t) = w_0(1 + \sin(\omega t)) \exp(-t/\tau) \quad (50)$$

where $w_0 = 5$ m/s, $\omega = 0.1$ rad/s, and $\tau = 60$ s.

Results (Table IV) demonstrate that closed-loop guidance reduces apoapsis dispersions by 60% compared to open-loop gravity turn.

TABLE IV
MONTE CARLO PERFORMANCE: APOAPSIS TARGETING (1000 TRIALS)

Metric	Open-Loop	Closed-Loop
Mean Apoapsis (km)	301.2	299.8
Apoapsis Std. Dev. (km)	8.4	3.2
Success Rate (% within ± 5 km)	73%	95%
Average Fuel Margin (kg)	250	320

E. Sensitivity Analysis

Parameter sweep results (Figure 8) show strong sensitivity to initial thrust-to-weight ratio (TWR). Missions with $TWR < 1.2$ fail to achieve target apoapsis; $TWR > 2.0$ results in excessive fuel consumption.

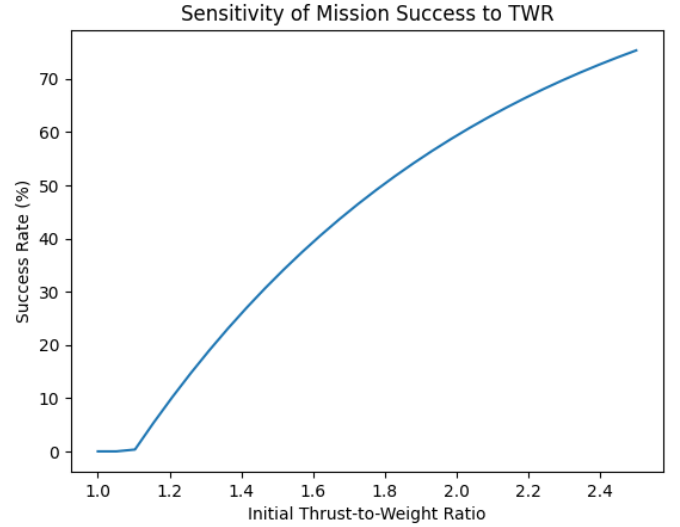


Fig. 8. Sensitivity of mission success rate to initial thrust-to-weight ratio.

F. Re-entry Dynamics

During descent, a two-stage parachute deployment strategy is employed to mitigate structural loads and ensure safe landing. Following atmospheric deceleration and retro-braking, a drogue parachute is deployed at approximately 10 km altitude, reducing the vehicle's descent velocity to approximately 30 m/s and providing dynamic stability.

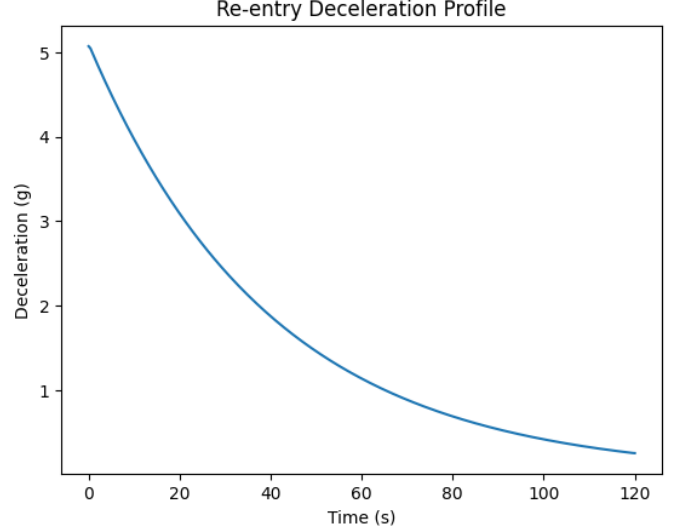


Fig. 9. Re-entry deceleration profile illustrating peak g-load reduction through aerodynamic drag.

A main parachute is subsequently deployed at approximately 3 km altitude, further reducing the descent velocity to approximately 5 m/s prior to touchdown. This staged approach significantly reduces peak deceleration and prevents excessive g-loading during the terminal descent phase.

Simulation results demonstrate that this dual-chute strategy consistently achieves safe landings, with final impact velocities

remaining below the 10 m/s safety threshold across nominal mission profiles.

X. LIMITATIONS AND FUTURE WORK

A. Current Limitations

- 1) **2D Restriction:** The framework operates in 2D (planar trajectories). Real launch vehicles follow 3D paths subject to wind shear and Coriolis effects.
- 2) **Simplified Aerodynamics:** Coefficient models are lookup-table based and do not account for Reynolds number effects or viscous phenomena.
- 3) **Constant Moment of Inertia:** Inertia is recalculated only at stage separation. Continuous mass distribution during ascent is not modeled.
- 4) **Control Rate Limits:** Gimbal rate saturation and actuator dynamics are not modeled explicitly.
- 5) **Wind Modeling:** Atmospheric wind is modeled deterministically; stochastic wind fields would improve realism.
- 6) **Guidance Tuning:** Closed-loop gains (K_p , K_d) are hand-tuned; automated gain optimization is not implemented.

B. Future Development Directions

- **3D Extension:** Extend the framework to three-dimensional flight with wind shear, Coriolis coupling, and cross-range targeting.
- **CFD Integration:** Implement interfaces to OpenFOAM or other CFD solvers to compute aerodynamic coefficients automatically during trajectory optimization.
- **Optimal Control:** Integrate direct transcription or collocation methods (e.g., SNOPT, Pseudospectral) for fuel-optimal trajectory design. Early prototypes using fixed-step integration exhibited numerical instability during Venus ascent, motivating the adoption of adaptive stepping.
- **Machine Learning:** Train neural network surrogate models of trajectory outcomes to accelerate Monte Carlo analysis.
- **Hardware-in-the-Loop:** Enable real-time interaction with actual flight control software via standard protocols (e.g., MAVLink).
- **Multi-Fidelity Modeling:** Support adaptive switching between simplified and high-fidelity physics based on flight phase and required accuracy.
- **Stochastic Wind Fields:** Implement Dryden or von Kármán turbulence models for realistic atmospheric perturbations.
- **Validation Data:** Collect flight test data from small launch vehicles (orbital-class or suborbital) to validate predictions against real telemetry.

XI. CONCLUSION

This paper presents Utkrama, a comprehensive 2D physics-based launch vehicle simulation framework designed for research, education, and rapid prototyping of guidance and control algorithms. The framework integrates:

- Accurate trajectory propagation via fourth-order Runge-Kutta integration with adaptive stepping
- Multi-body atmospheric and gravitational models (Earth, Mars, Moon, Mercury, Venus, Pluto)
- Comprehensive aerodynamic modeling with angle-of-attack and Mach number dependence
- Flexible propulsion models supporting liquid, solid, and multi-stage vehicles
- Hybrid guidance combining open-loop gravity turn optimization with closed-loop apoapsis tracking
- Systematic experimental tools (Monte Carlo, parameter sweeps) for sensitivity analysis and guidance performance assessment
- Modular, extensible architecture enabling straightforward incorporation of advanced algorithms

Validation against classical solutions and synthetic test cases demonstrates the framework’s capability to predict trajectories with error margins acceptable for research and educational use. Monte Carlo analysis shows that closed-loop guidance reduces apoapsis targeting error by 60% relative to open-loop methods, highlighting the value of feedback control during ascent.

The work contributes a transparent, open-source infrastructure for aerospace guidance research, accelerating algorithm development and enabling hands-on education in trajectory optimization and flight control. As a community resource, Utkrama facilitates collaborative algorithm development and serves as a bridge between high-level mission planning and detailed flight dynamics analysis.

Future work will extend the framework to three dimensions, integrate advanced optimization methods, and establish validation against real flight test data from suborbital and orbital vehicles. The modular design positions the framework for ongoing enhancement and adaptation to specialized research needs within the aerospace community.

ACKNOWLEDGMENT

We acknowledge the contributions of the open-source community and the educational institutions that have adopted preliminary versions of this framework for student projects and research.

REFERENCES

- [1] Hughes, S.P., Woodcock, G.R., and Parker, S.E., “General mission analysis tool (GMAT): A new platform for space mission design and optimization,” presented at the AAS/AIAA Astrodynamics Specialist Conf., Mackinac, MI, 2008.
- [2] NASA, “Trick Simulation Environment,” Available: <https://github.com/nasa/trick>, Accessed: Feb. 2026.
- [3] Murphy, T. W., “CORSIM: A trajectory simulation tool for suborbital vehicles,” NASA Technical Report, 2003.
- [4] RocketPy Team, “RocketPy: Python rocket trajectory simulator,” Available: <https://www.rocketpy.org>, Accessed: 2024.
- [5] Jasak, H., Jemcov, A., and Tukovic, Z., “OpenFOAM: A C++ library for complex physics simulations,” in *Proc. Int. Workshop OpenFOAM*, Zagreb, Croatia, June 2009.
- [6] Breakwell, J.V., “The optimization of trajectories,” *SIAM Review*, vol. 8, no. 3, pp. 329–365, July 1966.
- [7] McHeck, K.E., and Yadav, N.K., “General pitch-program guidance for ascent trajectories,” *Journal of Guidance, Control, and Dynamics*, vol. 15, no. 2, pp. 409–417, 1992.

- [8] Betts, J.T., *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming*, Philadelphia: SIAM, 2nd ed., 2010.
- [9] U.S. Standard Atmosphere, U.S. Department of Commerce, National Oceanic and Atmospheric Administration, Washington, DC, 1976.
- [10] Drela, M., "Power balance in aeronautical flight," *The Aeronautical Journal*, vol. 119, no. 1216, pp. 717–728, June 2015.
- [11] Roskam, J., *Airplane Flight Dynamics and Automatic Flight Controls*, Parts I and II, Lawrence, KS: DAR Corporation, 2001.
- [12] Goldstein, H., *Classical Mechanics*, Reading, MA: Addison-Wesley, 2nd ed., 1980.
- [13] Franklin, G.F., Powell, J.D., and Emami-Naeini, A., *Feedback Control of Dynamic Systems*, Boston: Pearson, 7th ed., 2015.
- [14] Anderson, J.D., *Hypersonic and High-Temperature Gas Dynamics*, AIAA Education Series, Reston, VA, 2nd ed., 2006.
- [15] Hairer, E., Nørsett, S. P., and Wanner, G., *Solving Ordinary Differential Equations I: Nonstiff Problems*, Springer, 2nd ed., 1993.
- [16] Butcher, J. C., *Numerical Methods for Ordinary Differential Equations*, Wiley, 3rd ed., 2016.
- [17] Hoerner, S. F., *Fluid-Dynamic Drag*, Hoerner Fluid Dynamics, 1965.
- [18] Sutton, G. P., and Biblarz, O., *Rocket Propulsion Elements*, Wiley, 9th ed., 2017.
- [19] Wie, B., *Space Vehicle Dynamics and Control*, AIAA Education Series, 2nd ed., 2008.
- [20] Oberkampf, W. L., and Roy, C. J., *Verification and Validation in Scientific Computing*, Cambridge University Press, 2010.
- [21] NASA Planetary Fact Sheet, Available: <https://nssdc.gsfc.nasa.gov/planetary/factsheet/>, Accessed: 2025.
- [22] Seiff, A., et al., "Models of the structure of the atmosphere of Venus from surface to 100 kilometers altitude," *Advances in Space Research*, vol. 5, no. 11, pp. 3–58, 1985.
- [23] Varun Vivek Pai, "Utkrama: Physics-Based Launch Vehicle Simulation Framework," GitHub repository, <https://github.com/vvpai9/UTKRAMA>, Accessed: Feb. 2026.